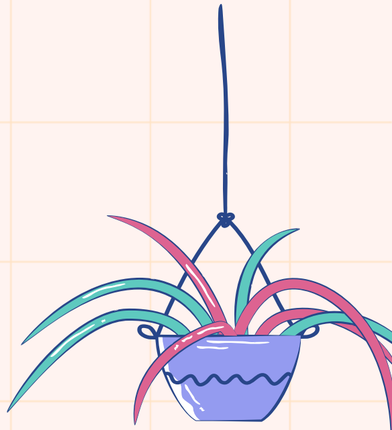
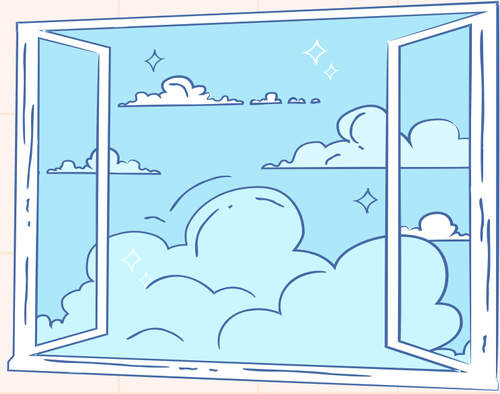
Generics in Java

Presented by Sara Abbasghorbani

Advanced Programming - Spring 2026 - Dr Vahidi Asl



What will we learn in this session?



1) Generics in Java

2) Generic Classes

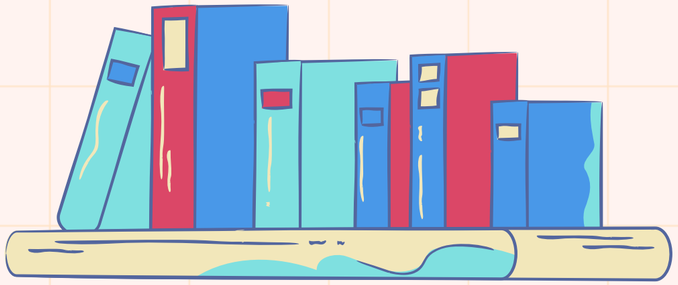
3) Interfaces and Generic

4) Generic Method

5) Erasure

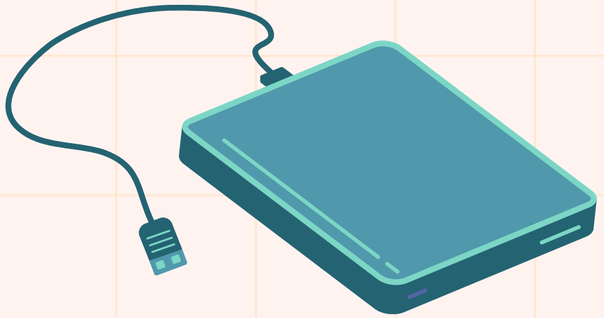
6) Limitations of Generic

Generics in Java



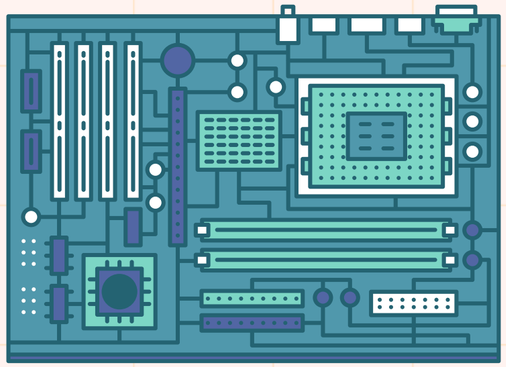
Generics in Java refer to parameterized types that allow writing code which works with multiple data types using a single class, interface, or method. They improve reusability and ensure type safety at compile time.

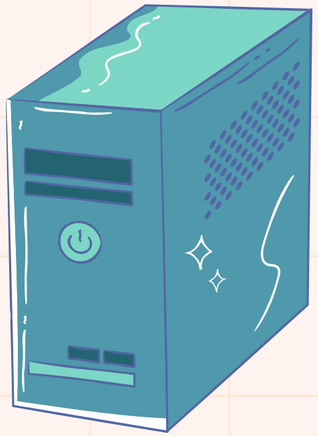
- Use type parameters (like <T>) to create flexible and reusable code
- Prevent runtime errors by enforcing type safety at compile time

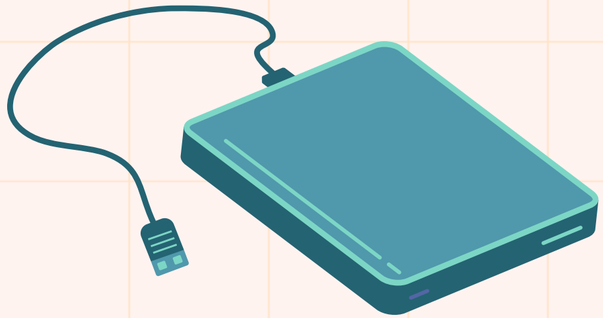


Why Use Generics?

- Before Generics, Java collections like `ArrayList` or `HashMap` could store any type of object, everything was treated as an Object. It had some problems.
- If you added a `String` to a `List`, Java didn't remember its type. You had to manually cast it when retrieving. If the type was wrong, it caused a runtime error.
- With Generics, you can specify the type the collection will hold like `ArrayList<String>`. Now, Java knows what to expect and it checks at compile time, not at runtime.







Generic Classes

A generic class is a class that can operate on objects of different types using a type parameter. Like C++, we use <> to specify parameter types in generic class creation.

Stack Class

- push(value)
- pop()

How can we use this class for each type value?

1

```java
public class Stack<E> {  2 usages
    private E[] elements ;  6 usages
    private int top ;  7 usages
    public void push(E pushValue){  3 usages
        if(top == elements.length - 1){
            throw new FullStackException("The Stack is Full") ;
        }
        else {
            elements[++top] = pushValue ;
        }
    }
    public E pop(){  1 usage
        if(top == -1){
            throw new EmptyStackException("The Stack is Empty") ;
        }
        return elements[top--] ;
    }
    public Stack(int maxsize){  1 usage
        top = -1 ;
        elements = (E[]) new Object[maxsize] ;
    }
```

2

```java
    public E[] getElements() {  no usages
        return elements;
    }

    public void setElements(E[] elements) {  no usages
        this.elements = elements;
    }

    public int getTop() {  no usages
        return top;
    }

    public void setTop(int top) {  no usages
        this.top = top;
    }
}
```

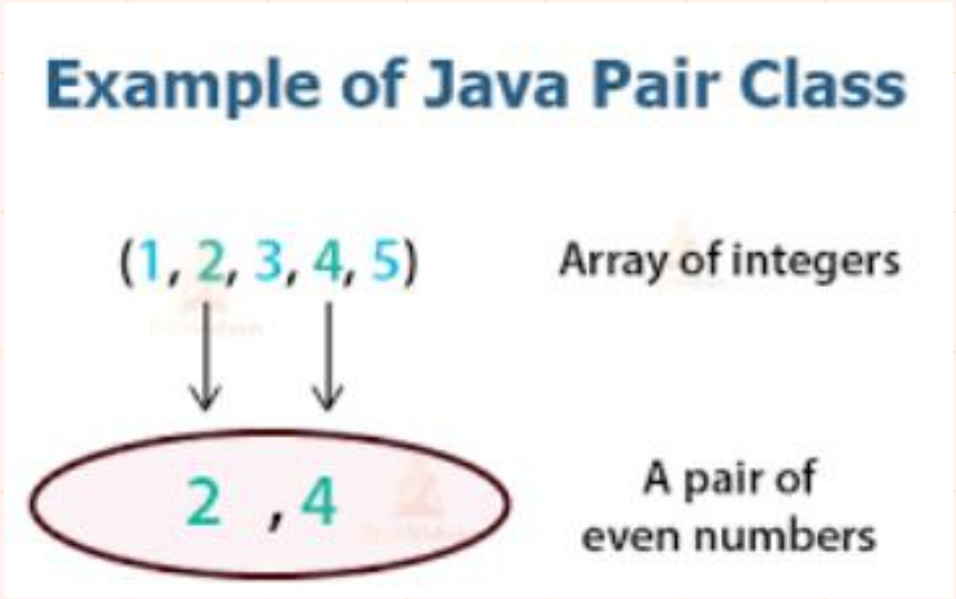
3

```java
class Stack_Test {
    public static void main(String[] args){
        Stack<String> stringStack = new Stack<>( maxsize: 10) ;
        stringStack.push( pushValue: "Sara");
        stringStack.push( pushValue: "Sima");
        stringStack.push( pushValue: "Sahar");
        System.out.println(stringStack.pop());
        Stack<Integer> integerStack = new Stack<>( maxsize: 10) ;
        integerStack.push( pushValue: 20);
        integerStack.push( pushValue: 9);
        integerStack.push( pushValue: 25);
        System.out.println(integerStack.pop());
    }
}
```

```
"C:\Program Files\Java\jdk-21\b
Sahar
25
```

How about Pair Class?

Pair Class what it does is it concatenates two values. These two values can be two integers, strings, or any other variable.



```java
public class Pair<T1, T2> {  no usages
    private T1 first ;  3 usages
    private T2 second ;  3 usages

    public Pair(T1 first, T2 second){  no usages
        this.first = first ;
        this.second = second ;
    }

    public T1 getFirst() {  no usages
        return first;
    }

    public void setFirst(T1 first) {  no usages
        this.first = first;
    }

    public T2 getSecond() {  no usages
        return second;
    }

    public void setSecond(T2 second) {  no usages
        this.second = second;
    }
}
```

```java
class Pair_Test {
    public static void main(String[] args){
        Pair<String, String> stringStringPair = new Pair<>("Nila" , "Mohammadi") ;
        System.out.println(stringStringPair.getFirst() + " " + stringStringPair.getSecond());
        Pair<String , Integer> stringIntegerPair = new Pair<>("Sina" , 20) ;
        String name = stringIntegerPair.getFirst();
        Integer age = stringIntegerPair.getSecond();
        System.out.println(name + " " + age);
    }
}
```

```
"C:\Program Files\Java\jdk-21\bi
Nila Mohammadi
Sina 20
```

Pairing Different Values (Generics)

The Pair<T1, T2> class uses Java Generics to group two related variables of any data type into a single object. It ensures code reusability and type safety (pairing a String name with an Integer age).

Interfaces and Generic

In Java, Generics and Interfaces work together powerfully to create flexible, reusable, and type-safe APIs. The primary relationship is that an interface itself can be generic.

A Generic Interface is an interface that is parameterized over one or more types. This means you define an interface with a type placeholder (like <T>), and the class that implements this interface specifies the actual concrete type.

1

```java
public interface Max_Min<T extends Comparable<T>> {
    T min() ;  1 usage  1 implementation
    T max() ;  1 usage  1 implementation
}
```

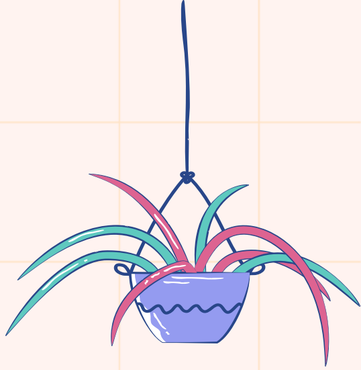
2

```java
class Compare<T extends Comparable<T>> implements Max_Min{
    T[] values ;  9 usages
    Compare(T[] obj){  1 usage
        values = obj ;
    }
    @Override  1 usage
    public T min() {
        T t1 = values[0];
        for(int i = 1 ; i < values.length ; i++){
            if(values[i].compareTo(t1) < 0){
                t1 = values[i];
            }
        }
        return t1 ;
    }

    @Override  1 usage
    public T max() {
        T t1 = values[0];
        for(int i = 1 ; i < values.length ; i++){
            if(values[i].compareTo(t1) > 0){
                t1 = values[i];
            }
        }
        return t1 ;
    }
}
```

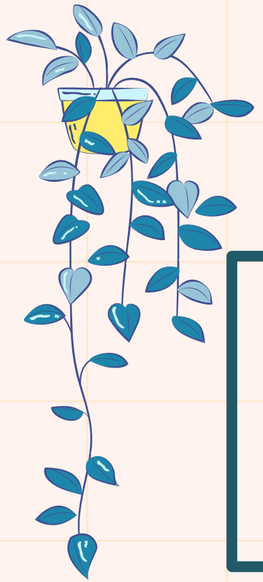
3

```java
class Max_Min_Test {
    public static void main(String[] args){
        Integer[] arr = {1, 4, 9, 2, 6, 3} ;
        Compare<Integer> integerCompare = new Compare<>(arr) ;
        System.out.println("Min Value : " + integerCompare.min());
        System.out.println("Max value : " + integerCompare.max());
    }
}
```

```
"C:\Program Files\Jav
Min Value : 1
Max value : 9
```



A few notes about Generic data types

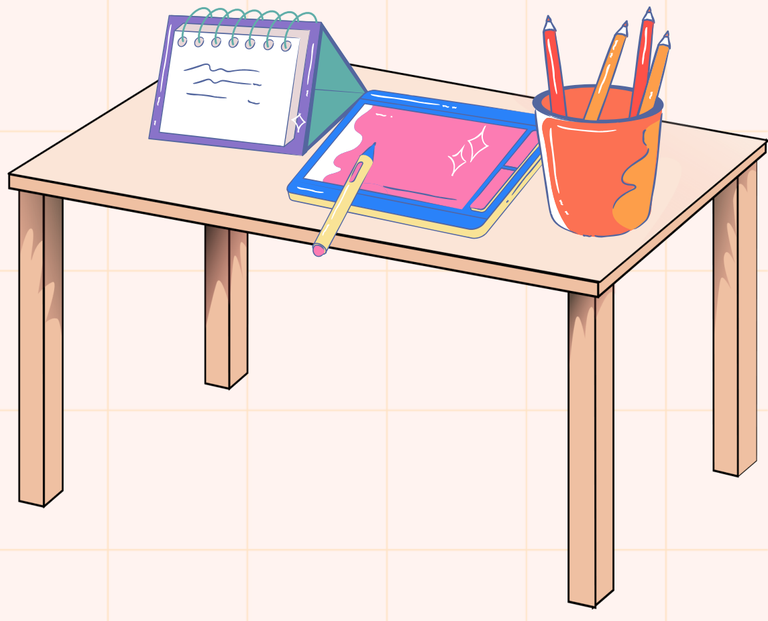


Specifying Generic Types

When declaring a variable of a generic type, you can specify the generic parameter with a concrete type.

For example, consider the List interface:

```
List<String> strs;
List<Integer> ints;
List<Student> stus;
```

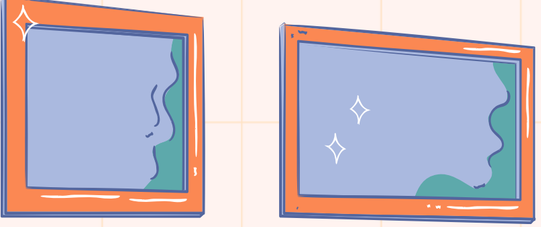


⚠️You can't use primitive data types (like int, double) as type parameters.

The type substituted for E must be a Class (Reference Type).

Compile Error:

```
List<int> E1;
List<double> E2;
```



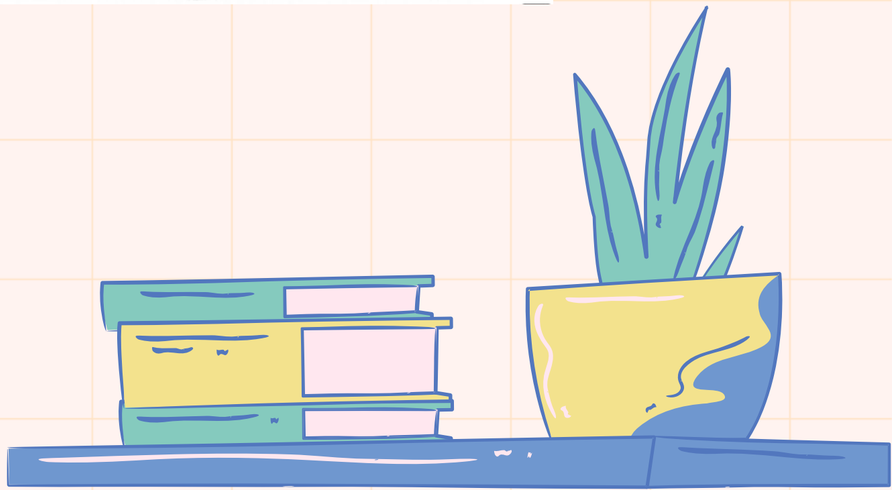
Benefits of Using Generics

1) Stronger Type-Checking at Compile Time

2) Catches errors early during compilation rather than at runtime.

3) Prevents unexpected `ClassCastException` errors.

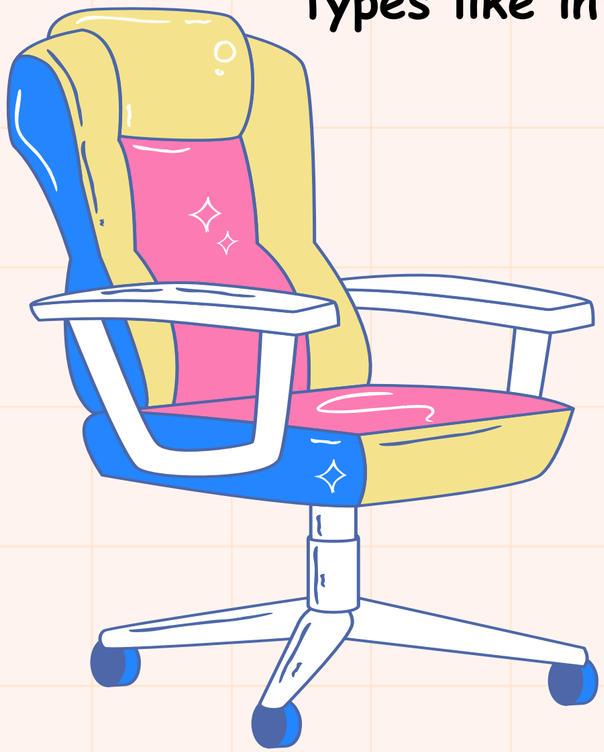
4) Results in cleaner, less, and more readable code.



⚠ Limitations of Generics

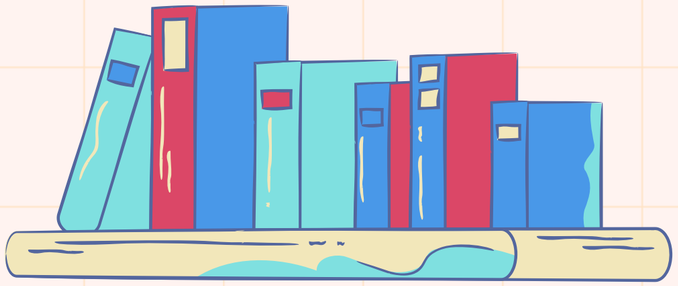
1.Generics Work Only with Reference Types:

When we declare an instance of a generic type, the type argument passed to the type parameter must be a reference type. We cannot use primitive data types like int, char.

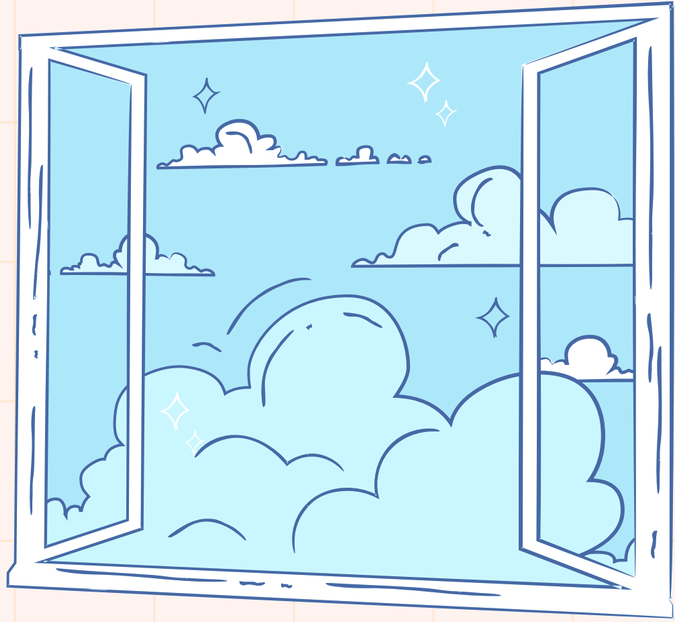


2. Generic Types Differ Based on their Type Arguments

Generic types differ based on their type arguments, but this difference exists only at compile time. During compilation, Java removes generic type information through a process called type erasure, replacing type parameters with their bounds or Object. As a result, generics ensure type safety at compile time while maintaining backward compatibility at runtime.

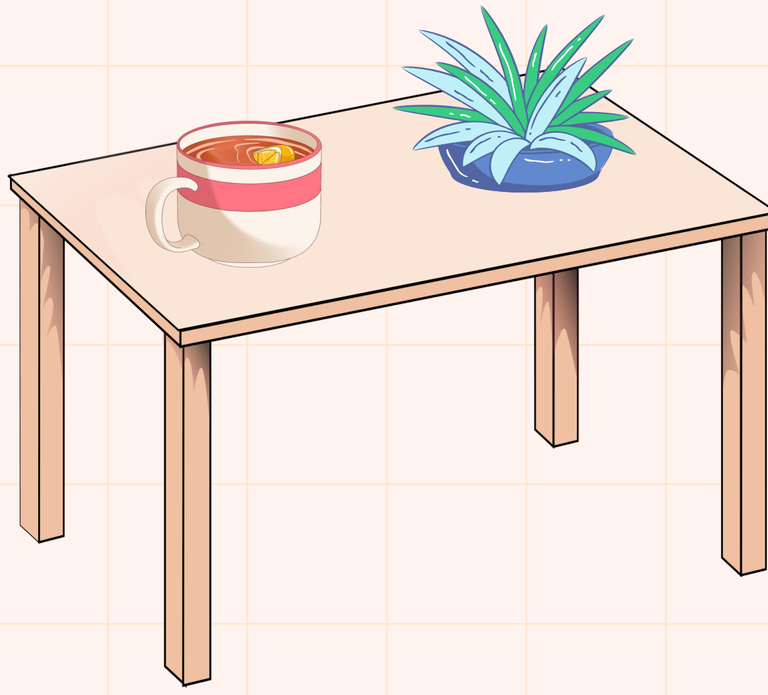


Generic Method



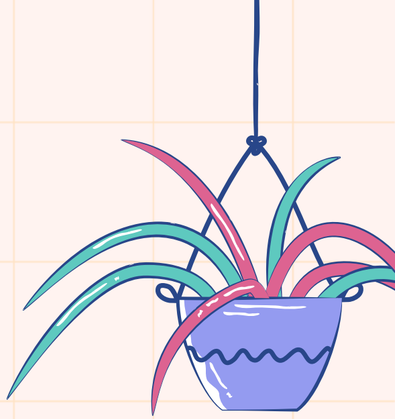
We can also write generic methods that can be called with different types of arguments based on the type of arguments passed to the generic method. The compiler handles each method.

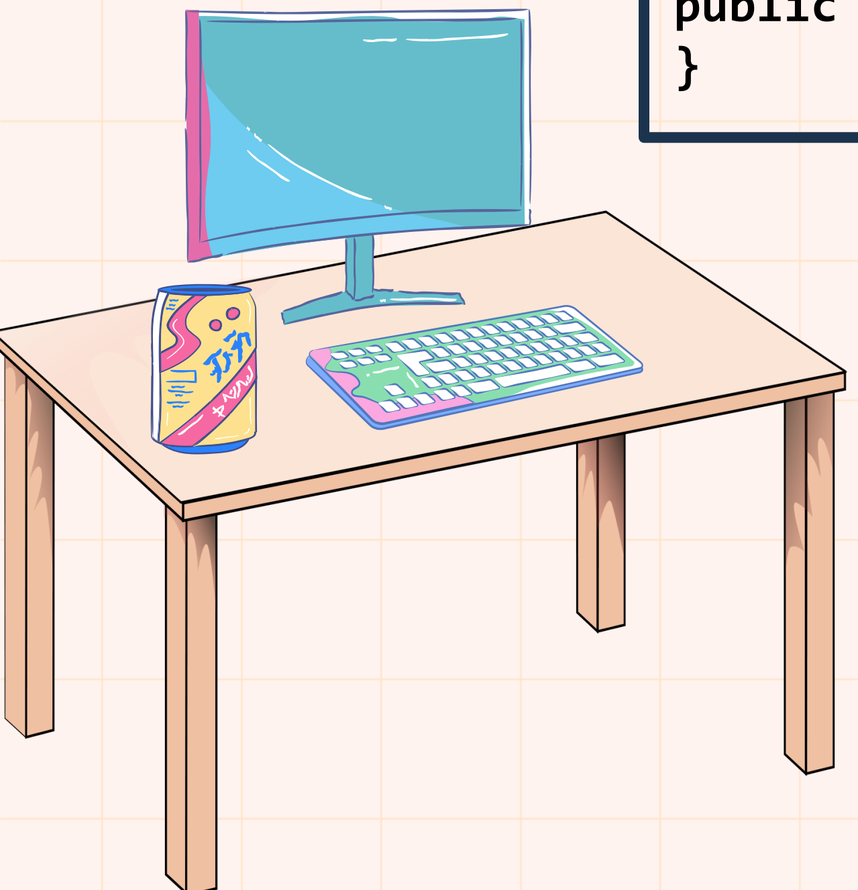
```
    static <T> void genericDisplay(T element){  no usages  new *
        System.out.println(element.getClass().getName() + " = " + element);
    }
}
```



GenericClass<T> and NotGenericClass

Both can have Generic methods.

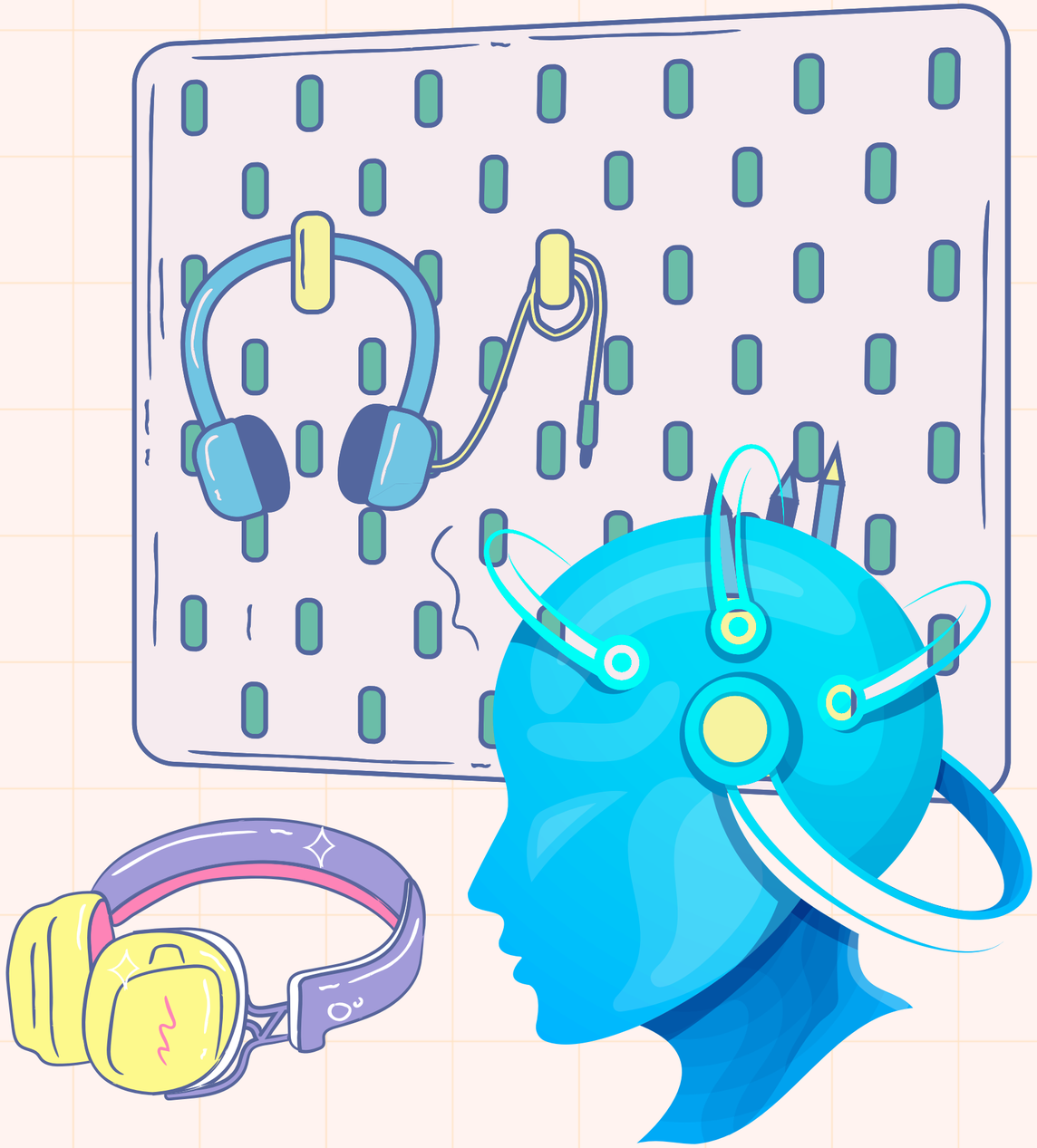


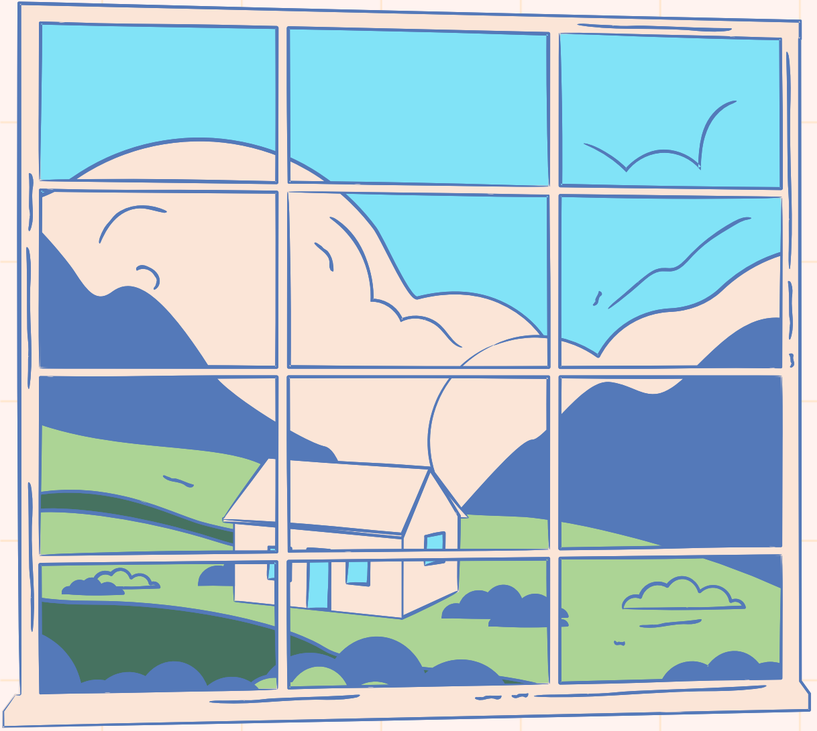


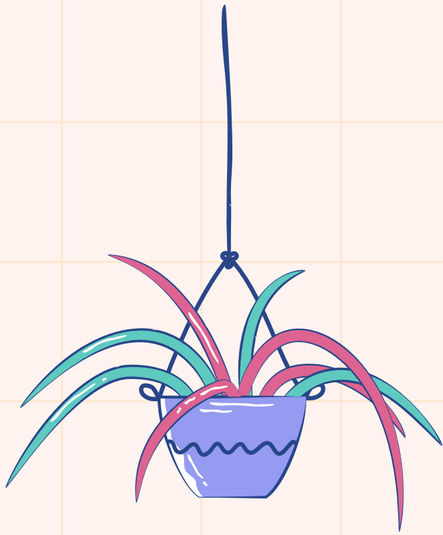
```
class GenericClass{
public void f(E p1, T p2){}
}
```

```
class NotGeneric{
public T chooseRandom(T p1, T p2){
if(new Random().nextFloat()>0.5){
return p1;
}
 return p2;
}
public static E max(E p1, E p2){
return p1.compareTo(p2) > 0 ? p1 : p2;
 }
}
```

Erasure



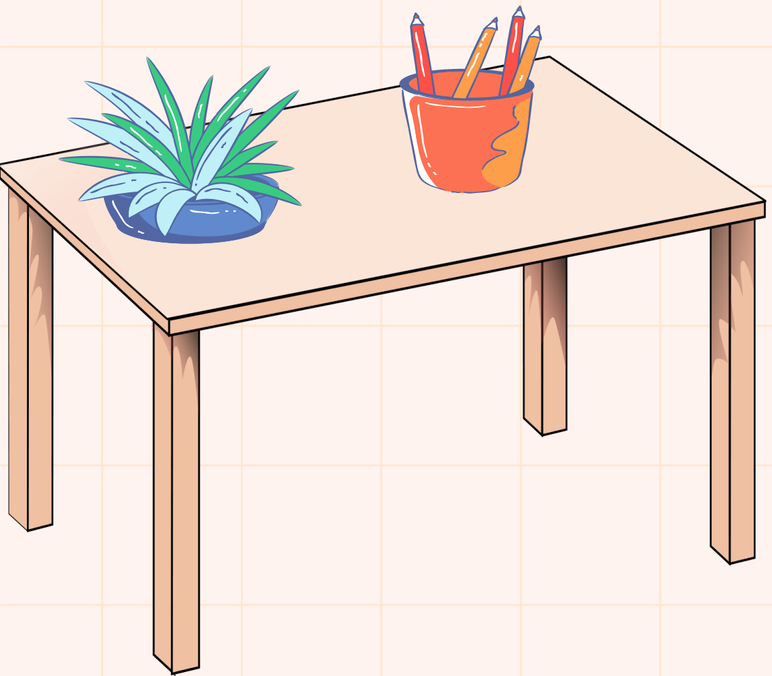


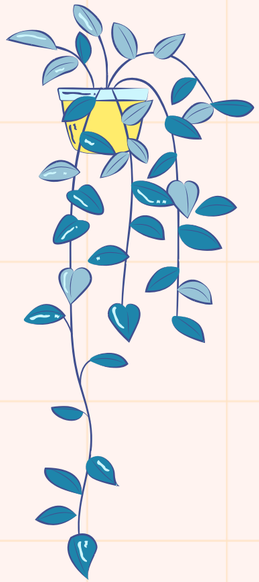


Type Erasure is the process by which the Java compiler removes all generic type information and replaces type parameters with their upper bounds (or Object if unbounded).

During compilation, the compiler performs:

- Replacing type parameters with their upper bound or Object.
- Inserting casts where needed to maintain type safety.
- Generating bridge methods to preserve method overriding in generic hierarchies.





How Type Erasure Works

1. Unbounded Type Example

```
class Stack{
void push(T s){}
T pop() {…}
 }
```

→

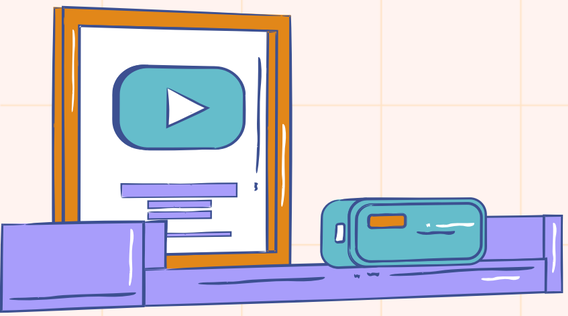
```
class Stack{
void push(Object s){}
Object pop() {…}
  }
```

2. Bounded Type Example

```
class SortedList<T extends Comparable<T>>{
T[] values;
public void add(T o){…}
 }
```

→

```
class SortedList{
Comparable[] values;
public void add(Comparable o){…}
 }
```

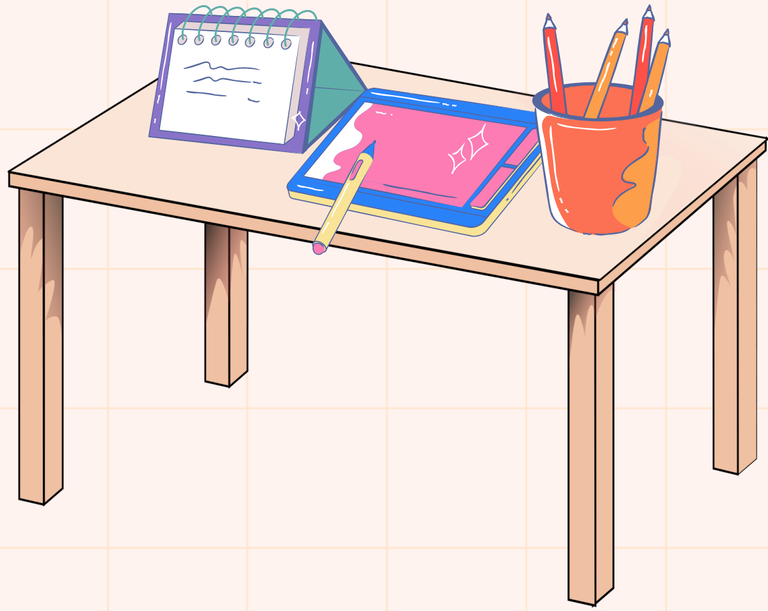


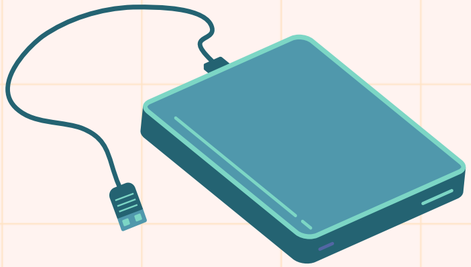
Type Erasure in Action(Raw Types)

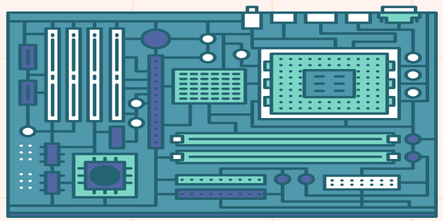
```java
import java.util.*;

public class Geeks{

    public static void main(String args[]){

        List list = new ArrayList();   // Raw type
        list.add("Hello");

        String s;
        for (Iterator iter = list.iterator(); iter.hasNext(); ) {
            s = (String) iter.next();
            System.out.println(s);
        }
    }
}
```

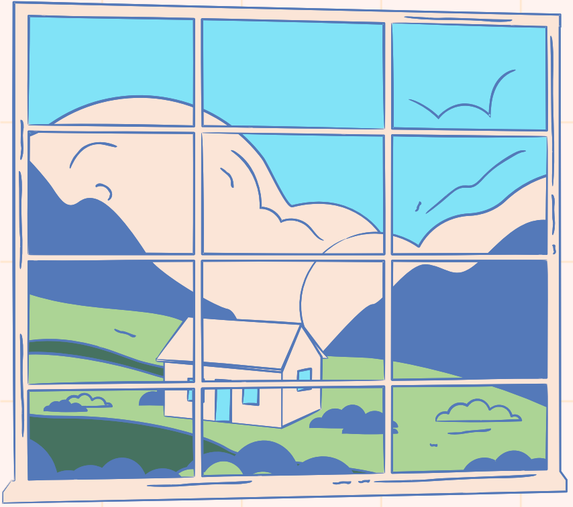
```
"C:\Program Files\J
Hello
```

Explanation: In the above example, we are using raw types, it simply means we are not specifying the type if datat the the list will hold. In this case, the compiler will throw a warning GenericsErasure.java uses unchecked or unsafe operations. The reason for the warning is that raw types don not carry any type information.

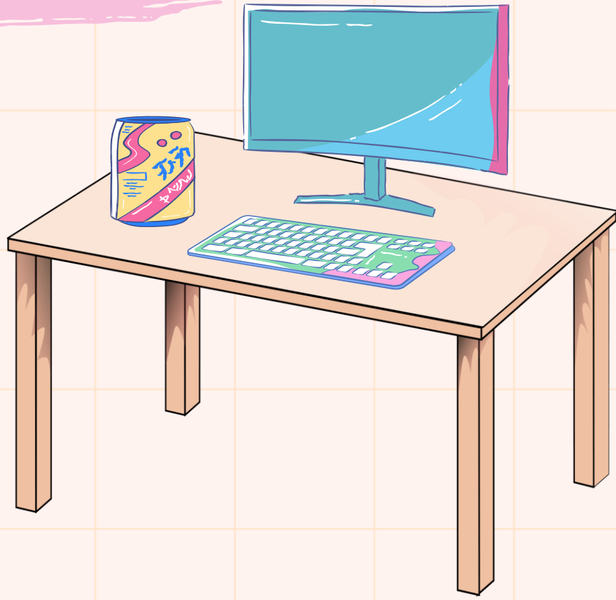


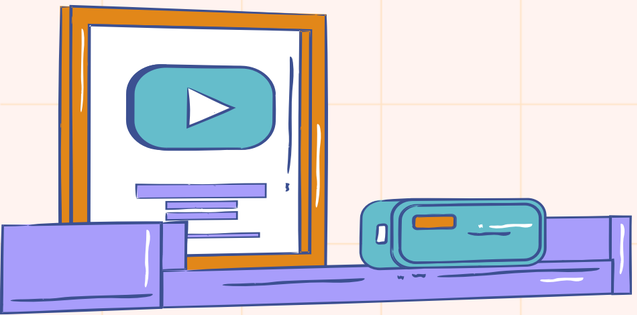






Limitations of Generic data type





Java Generics are a powerful feature, enabling developers to write flexible, reusable, and type-safe code. By allowing classes, interfaces, and methods to operate on objects of various types while ensuring type safety, generics have become an indispensable part of Java development. However, generics are not without their limitations, and understanding these constraints is essential to using them effectively.

Type Erasure: The Root Cause of Most Limitations

The most significant limitation of Java Generics stems from type erasure.

```java
public class Box<T> {
    private T value;

    public void setValue(T value) {
        this.value = value;
    }

    public T getValue() {
        return value;
    }
}
```

Type Erasure

```java
public class Box {
    private Object value;

    public void setValue(Object value) {
        this.value = value;
    }

    public Object getValue() {
        return value;
    }
}
```

```java
Box<String> stringBox = new Box<>();

stringBox.setValue("Hello");

String value = stringBox.getValue();
```

Type Erasure

```java
Box stringBox = new Box();

stringBox.setValue("Hello");

String value = (String) stringBox.getValue();
```

⚠ Limitations of Java Generics

1. No Runtime Type Information: You cannot check the type of a generic object at runtime.

This means no instanceof checks or reflective type checks with generic types.

```
List<String> stringList = new ArrayList<>();

if (stringList instanceof List<String>) { //
Error: illegal generic type for instanceof
// ...
}
```

At runtime, stringList is simply List. The type information (String) is erased, and the compiled code treats it as List<Object>.

⚠ Limitations of Java Generics

2. Cannot Instantiate Generic Types with Primitive Types: Java generics do not support primitive types like int, char, or double. This is because generics are implemented using type erasure. As a result, you cannot directly use primitive types with generics. Instead, you must use their corresponding wrapper classes, such as Integer, Character, and Double.

3. Cannot Create Instances of Type Parameters: You cannot create an instance of a generic type parameter because the type parameter is erased at runtime. The Java compiler does not know what type T will be, so it cannot instantiate it.

```
public class MyClass<T> {
T instance = new T(); // Error: Type
parameter 'T' cannot be instantiated directly
}
```

⚠ Limitations of Java Generics

4. Cannot Declare Static Fields with Type Parameters: Static fields are shared among all instances of a class, but generic type parameters are specific to each instance. Therefore, you cannot declare a static field with a generic type parameter.

```
public class MyClass<T> {
private static T instance; // 'MyClass.this' cannot
be referenced from a static context
}
```

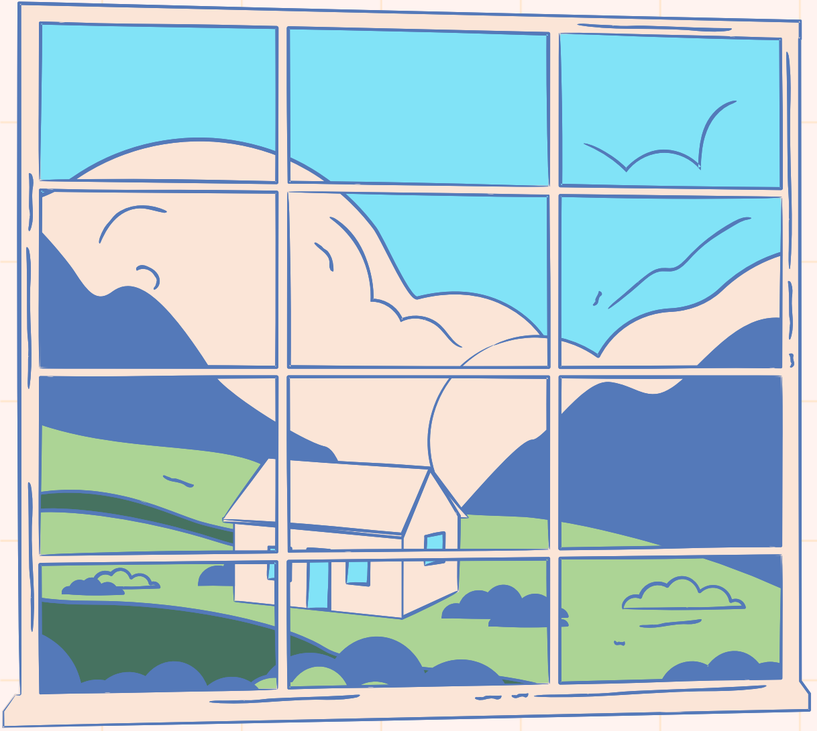
5. Cannot Use Casts or instanceof with Parameterized Types: Due to type erasure, you cannot use the instanceof operator with parameterized types. The type information is not available at runtime, making such checks unreliable.

Wrong

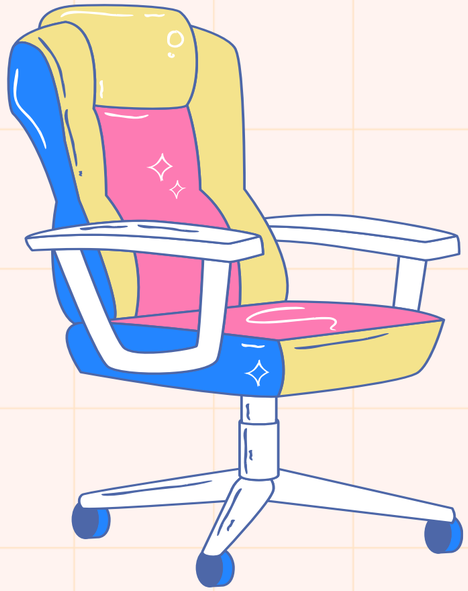
```
if (obj instanceof
List<String>) { // Compile-time
error
// ...
}
```

Correct

```
if (obj instanceof
List<?>) { // Correct
// ...
}
```



While Java Generics provide powerful tools for writing reusable, type-safe code, they come with a range of limitations. Understanding these limitations is crucial for Java developers, as it allows you to design better solutions and avoid common pitfalls. While generics offer significant benefits, such as reducing code duplication and ensuring type safety, they also require careful consideration to avoid unexpected errors or inefficiencies.

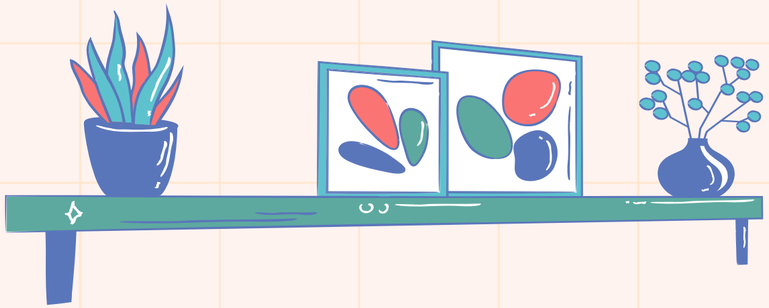


Resources

Limitations of Java Generics

Generic Classes and Erasure

Generics in Java



Thank You